MS SQL Server Basic Interview Questions and Answers

Table of Contents

SQL Fundamentals

1. What is SQL and MS SQL Server?

Answer:

SQL (Structured Query Language):

- Standard language for managing relational databases
- Used to create, read, update, and delete (CRUD) database records
- Pronounced as “S-Q-L” or “Sequel”

MS SQL Server:

- Relational Database Management System (RDBMS) developed by Microsoft
- Runs on Windows and Linux
- Uses T-SQL (Transact-SQL) - Microsoft’s extension of SQL
- Enterprise-grade database solution

Key Features:

- Data storage and retrieval
- Transaction management
- Security and user management
- Backup and recovery

- Replication and high availability
- Business intelligence tools

---

2. What are the different types of SQL commands?

Answer:

Category	Purpose	Commands
DDL (Data Definition Language)	Define database structure	CREATE, ALTER, DROP, TRUNCATE
DML (Data Manipulation Language)	Manipulate data	SELECT, INSERT, UPDATE, DELETE
DCL (Data Control Language)	Control access	GRANT, REVOKE
TCL (Transaction Control Language)	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

Examples:

```
-- DDL
CREATE TABLE Employees (EmployeeID INT, Name VARCHAR(50));
ALTER TABLE Employees ADD Email VARCHAR(100);
DROP TABLE Employees;

-- DML
SELECT * FROM Employees;
INSERT INTO Employees VALUES (1, 'John');
UPDATE Employees SET Name = 'Jane' WHERE EmployeeID = 1;
DELETE FROM Employees WHERE EmployeeID = 1;

-- DCL
GRANT SELECT ON Employees TO UserName;
REVOKE SELECT ON Employees FROM UserName;

-- TCL
BEGIN TRANSACTION;
    UPDATE Accounts SET Balance = Balance - 100 WHERE AccountID = 1;
    UPDATE Accounts SET Balance = Balance + 100 WHERE AccountID = 2;
COMMIT;
```

---

3. What is the difference between SQL and T-SQL?

Answer:

Feature	SQL	T-SQL (Transact-SQL)
Standard	ANSI/ISO standard	Microsoft extension
Scope	Generic	SQL Server specific
Programming	Limited	Supports variables, loops, error handling
Procedural	No	Yes (IF, WHILE, TRY-CATCH)

T-SQL Specific Features:

```
-- Variables
DECLARE @EmployeeName VARCHAR(50);
SET @EmployeeName = 'John Doe';
```

```
-- Control Flow
IF @EmployeeName = 'John Doe'
BEGIN
    PRINT 'Employee found';
END
ELSE
BEGIN
    PRINT 'Employee not found';
END

-- WHILE Loop
DECLARE @Counter INT = 1;
WHILE @Counter <= 10
BEGIN
    PRINT @Counter;
    SET @Counter = @Counter + 1;
END

-- Error Handling
BEGIN TRY
    -- Some operation
    SELECT 1/0;
END TRY
BEGIN CATCH
    PRINT 'Error occurred:  ' + ERROR_MESSAGE();
END CATCH
```

Database Basics

4. How do you create and drop a database?

Answer:

```
-- Create Database
CREATE DATABASE CompanyDB;

-- Create Database with options
CREATE DATABASE CompanyDB
ON PRIMARY
(
    NAME = 'CompanyDB_Data',
    FILENAME = 'C:\Data\CompanyDB. mdf',
    SIZE = 10MB,
    MAXSIZE = 100MB,
    FILEGROWTH = 5MB
)
LOG ON
(
    NAME = 'CompanyDB_Log',
    FILENAME = 'C:\Data\CompanyDB.ldf',
    SIZE = 5MB,
    MAXSIZE = 50MB,
    FILEGROWTH = 5MB
);

-- Use Database
USE CompanyDB;

-- Check current database
```

```
SELECT DB_NAME();

-- List all databases
SELECT name FROM sys.databases;

-- Drop Database
DROP DATABASE CompanyDB;

-- Drop Database if exists (SQL Server 2016+)
DROP DATABASE IF EXISTS CompanyDB;
```

---

5. What is a schema in SQL Server?

Answer:

A schema is a logical container for database objects (tables, views, procedures, etc.).

Benefits:

- Organize database objects
- Apply security at schema level
- Avoid naming conflicts
- Logical grouping

```
-- Create Schema
CREATE SCHEMA Sales;
CREATE SCHEMA HR;

-- Create table in schema
CREATE TABLE Sales.Orders
(
    OrderID INT PRIMARY KEY,
    OrderDate DATE,
    Amount DECIMAL(10,2)
);

CREATE TABLE HR.Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    Department VARCHAR(50)
);

-- Query from schema
SELECT * FROM Sales. Orders;
SELECT * FROM HR. Employees;

-- Default schema is 'dbo'
SELECT * FROM dbo.MyTable;

-- Change schema of table
ALTER SCHEMA Sales TRANSFER dbo.OldTable;

-- Drop Schema (must be empty)
DROP SCHEMA Sales;
```

Tables and Data Types

6. What are the common data types in SQL Server?

Answer:

Numeric Types:

Data Type	Range	Storage	Description
INT	-2,147,483,648 to 2,147,483,647	4 bytes	Standard integer
BIGINT	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	8 bytes	Large integer
SMALLINT	-32,768 to 32,767	2 bytes	Small integer
TINYINT	0 to 255	1 byte	Very small integer
DECIMAL(p,s)	Varies	5-17 bytes	Fixed precision
FLOAT	Varies	4 or 8 bytes	Floating point

String Types:

Data Type	Max Length	Storage	Description
VARCHAR(n)	1 to 8,000	Variable	Variable-length string
NVARCHAR(n)	1 to 4,000	Variable	Unicode string
CHAR(n)	1 to 8,000	Fixed	Fixed-length string
TEXT	2GB	Variable	Large text (deprecated)

Date and Time Types:

Data Type	Format	Range
DATE	YYYY-MM-DD	0001-01-01 to 9999-12-31
TIME	HH:MM:SS	00:00:00 to 23:59:59.9999999
DATETIME	YYYY-MM-DD HH: MM:SS	1753-01-01 to 9999-12-31
DATETIME2	YYYY-MM-DD HH:MM:SS	0001-01-01 to 9999-12-31
SMALLDATETIME	YYYY-MM-DD HH:MM:SS	1900-01-01 to 2079-06-06

Other Types:

```
-- Binary
BINARY(n)      -- Fixed-length binary
VARBINARY(n)   -- Variable-length binary
IMAGE          -- Large binary (deprecated)

-- Boolean
BIT            -- 0, 1, or NULL

-- Unique Identifier
UNIQUEIDENTIFIER  -- GUID

-- XML
XML            -- XML data

-- JSON (stored as NVARCHAR)
```

```
NVARCHAR(MAX)  -- JSON data
```

Example:

```
CREATE TABLE Products
(
    ProductID INT,
    ProductName VARCHAR(100),
    Price DECIMAL(10,2),
    InStock BIT,
    CreatedDate DATETIME,
    Description NVARCHAR(MAX)
);
```

---

7. How do you create, alter, and drop a table?

Answer:

```
-- CREATE TABLE
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY IDENTITY(1,1),
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    HireDate DATE DEFAULT GETDATE(),
    Salary DECIMAL(10,2),
    DepartmentID INT,
    IsActive BIT DEFAULT 1
);

-- CREATE TABLE with Foreign Key
CREATE TABLE Departments
(
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100) NOT NULL
);

CREATE TABLE Employees2
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100),
    DepartmentID INT FOREIGN KEY REFERENCES Departments(DepartmentID)
);

-- CREATE TABLE from another table
SELECT * INTO EmployeesBackup FROM Employees;

-- CREATE TABLE with only structure (no data)
SELECT * INTO EmployeesEmpty FROM Employees WHERE 1=0;

-- ALTER TABLE - Add Column
ALTER TABLE Employees ADD PhoneNumber VARCHAR(20);

-- ALTER TABLE - Modify Column
ALTER TABLE Employees ALTER COLUMN PhoneNumber VARCHAR(15);

-- ALTER TABLE - Drop Column
ALTER TABLE Employees DROP COLUMN PhoneNumber;
```

```
-- ALTER TABLE - Add Constraint
ALTER TABLE Employees
ADD CONSTRAINT FK_Employees_Departments
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID);

-- ALTER TABLE - Drop Constraint
ALTER TABLE Employees DROP CONSTRAINT FK_Employees_Departments;

-- DROP TABLE
DROP TABLE Employees;

-- DROP TABLE if exists
DROP TABLE IF EXISTS Employees;

-- TRUNCATE TABLE (removes all data, keeps structure)
TRUNCATE TABLE Employees;

-- Check if table exists
IF OBJECT_ID('Employees', 'U') IS NOT NULL
    PRINT 'Table exists';
```

SELECT Statement

8. What is the basic syntax of SELECT statement?

Answer:

```
-- Basic Syntax
SELECT column1, column2, ...
FROM table_name
WHERE condition
ORDER BY column1, column2, ...  [ASC|DESC];

-- SELECT all columns
SELECT * FROM Employees;

-- SELECT specific columns
SELECT FirstName, LastName, Email FROM Employees;

-- SELECT with alias
SELECT FirstName AS 'First Name',
       LastName AS 'Last Name',
       Salary * 12 AS 'Annual Salary'
FROM Employees;

-- SELECT DISTINCT (unique values)
SELECT DISTINCT Department FROM Employees;

-- SELECT TOP
SELECT TOP 10 * FROM Employees;

-- SELECT TOP with PERCENT
SELECT TOP 10 PERCENT * FROM Employees;

-- SELECT with ORDER BY
SELECT * FROM Employees ORDER BY LastName ASC;
SELECT * FROM Employees ORDER BY Salary DESC;

-- SELECT with multiple ORDER BY
```

```
SELECT * FROM Employees
ORDER BY Department ASC, Salary DESC;
```

---

9. How do you use wildcards in SQL Server?

Answer:

```
-- % - Represents zero or more characters
SELECT * FROM Employees WHERE LastName LIKE 'S%';      -- Starts with S
SELECT * FROM Employees WHERE LastName LIKE '%son';    -- Ends with son
SELECT * FROM Employees WHERE LastName LIKE '%and%';   -- Contains and

-- _ - Represents a single character
SELECT * FROM Employees WHERE FirstName LIKE 'J_hn';   -- John, Jahn, etc.
SELECT * FROM Employees WHERE Phone LIKE '555-____';   -- 555- followed by 4 digits

-- [] - Any single character within brackets
SELECT * FROM Employees WHERE LastName LIKE '[ABC]%';  -- Starts with A, B, or C
SELECT * FROM Employees WHERE Code LIKE '[0-9][0-9]';  -- Two digits

-- [^] - Any character NOT in brackets
SELECT * FROM Employees WHERE LastName LIKE '[^ABC]%'; -- NOT starts with A, B, or C

-- Escape special characters
SELECT * FROM Products WHERE Description LIKE '%30[%]%'; -- Contains "30%"

-- Examples
-- Find names starting with 'A' or 'J'
SELECT * FROM Employees WHERE FirstName LIKE '[AJ]%';

-- Find phone numbers with pattern
SELECT * FROM Employees WHERE Phone LIKE '___-___-____';

-- Find emails from specific domains
SELECT * FROM Employees WHERE Email LIKE '%@gmail.com' OR Email LIKE '%@yahoo. com';
```

WHERE Clause and Filtering

10. What are the different operators used in WHERE clause?

Answer:

Comparison Operators:

```
-- Equal to
SELECT * FROM Employees WHERE DepartmentID = 5;

-- Not equal to
SELECT * FROM Employees WHERE DepartmentID <> 5;
SELECT * FROM Employees WHERE DepartmentID != 5;

-- Greater than / Less than
SELECT * FROM Employees WHERE Salary > 50000;
SELECT * FROM Employees WHERE Age < 30;

-- Greater than or equal to / Less than or equal to
SELECT * FROM Employees WHERE Salary >= 50000;
```

```
SELECT * FROM Employees WHERE Age <= 30;
```

Logical Operators:

```
-- AND
SELECT * FROM Employees
WHERE Department = 'IT' AND Salary > 60000;

-- OR
SELECT * FROM Employees
WHERE Department = 'IT' OR Department = 'HR';

-- NOT
SELECT * FROM Employees
WHERE NOT Department = 'IT';

-- Combined
SELECT * FROM Employees
WHERE (Department = 'IT' OR Department = 'HR')
AND Salary > 50000;
```

Special Operators:

```
-- BETWEEN
SELECT * FROM Employees
WHERE Salary BETWEEN 40000 AND 60000;

-- IN
SELECT * FROM Employees
WHERE Department IN ('IT', 'HR', 'Finance');

-- IS NULL / IS NOT NULL
SELECT * FROM Employees WHERE ManagerID IS NULL;
SELECT * FROM Employees WHERE Email IS NOT NULL;

-- LIKE (pattern matching)
SELECT * FROM Employees WHERE LastName LIKE 'S%';

-- EXISTS
SELECT * FROM Employees e
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o. EmployeeID = e.EmployeeID);
```

Examples:

```
-- Find employees in IT department with salary > 50000
SELECT * FROM Employees
WHERE Department = 'IT' AND Salary > 50000;

-- Find employees hired in 2020
SELECT * FROM Employees
WHERE HireDate BETWEEN '2020-01-01' AND '2020-12-31';

-- Find employees without manager
SELECT * FROM Employees WHERE ManagerID IS NULL;

-- Find employees with names starting with A, B, or C
SELECT * FROM Employees
WHERE LastName LIKE '[ABC]%';
```

Joins

11. What are the different types of JOINs in SQL Server?

Answer:

```
-- Sample Tables
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    DepartmentID INT
);

CREATE TABLE Departments
(
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);

-- Insert Sample Data
INSERT INTO Employees VALUES (1, 'John', 1), (2, 'Jane', 2), (3, 'Bob', NULL);
INSERT INTO Departments VALUES (1, 'IT'), (2, 'HR'), (3, 'Finance');
```

1. INNER JOIN - Returns matching rows from both tables

```
SELECT e.Name, d.DepartmentName
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d. DepartmentID;

-- Result: John (IT), Jane (HR)
-- Bob excluded (no department), Finance excluded (no employees)
```

2. LEFT JOIN (LEFT OUTER JOIN) - All rows from left table + matching from right

```
SELECT e.Name, d.DepartmentName
FROM Employees e
LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID;

-- Result: John (IT), Jane (HR), Bob (NULL)
-- All employees included, Finance excluded
```

3. RIGHT JOIN (RIGHT OUTER JOIN) - All rows from right table + matching from left

```
SELECT e.Name, d.DepartmentName
FROM Employees e
RIGHT JOIN Departments d ON e.DepartmentID = d.DepartmentID;

-- Result: John (IT), Jane (HR), NULL (Finance)
-- All departments included, Bob excluded
```

4. FULL OUTER JOIN - All rows from both tables

```
SELECT e.Name, d.DepartmentName
FROM Employees e
FULL OUTER JOIN Departments d ON e.DepartmentID = d.DepartmentID;

-- Result: John (IT), Jane (HR), Bob (NULL), NULL (Finance)
-- All employees and all departments included
```

5. CROSS JOIN - Cartesian product (all possible combinations)

```
SELECT e.Name, d.DepartmentName
FROM Employees e
CROSS JOIN Departments d;

-- Result: Every employee paired with every department (3 x 3 = 9 rows)
```

6. SELF JOIN - Join table with itself

```
CREATE TABLE Employees2
(
    EmployeeID INT,
    Name VARCHAR(50),
    ManagerID INT
);

INSERT INTO Employees2 VALUES (1, 'John', NULL), (2, 'Jane', 1), (3, 'Bob', 1);

-- Find employees and their managers
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees2 e
LEFT JOIN Employees2 m ON e.ManagerID = m. EmployeeID;

-- Result:
-- John (NULL) - no manager
-- Jane (John)
-- Bob (John)
```

Visual Representation:

```
INNER JOIN:       Only matching rows
LEFT JOIN:       All from left + matching from right
RIGHT JOIN:      All from right + matching from left
FULL OUTER JOIN: All from both tables
CROSS JOIN:      All possible combinations
```

---

12. Write queries using multiple joins

Answer:

```
-- Sample Schema
CREATE TABLE Customers
(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    CityID INT
);

CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,
    OrderDate DATE,
    CustomerID INT,
    EmployeeID INT
);

CREATE TABLE OrderDetails
(
```

```
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT,
    UnitPrice DECIMAL(10,2)
);

CREATE TABLE Products
(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100),
    CategoryID INT
);

CREATE TABLE Categories
(
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(50)
);

CREATE TABLE Cities
(
    CityID INT PRIMARY KEY,
    CityName VARCHAR(50)
);

-- Query 1: Get order details with product and category information
SELECT
    o.OrderID,
    o.OrderDate,
    p.ProductName,
    c. CategoryName,
    od.Quantity,
    od. UnitPrice,
    od. Quantity * od.UnitPrice AS TotalPrice
FROM Orders o
INNER JOIN OrderDetails od ON o.OrderID = od.OrderID
INNER JOIN Products p ON od.ProductID = p.ProductID
INNER JOIN Categories c ON p.CategoryID = c.CategoryID;

-- Query 2: Get customer orders with city information
SELECT
    cu.CustomerName,
    ci.CityName,
    o.OrderID,
    o.OrderDate,
    SUM(od.Quantity * od.UnitPrice) AS OrderTotal
FROM Customers cu
INNER JOIN Cities ci ON cu.CityID = ci.CityID
INNER JOIN Orders o ON cu.CustomerID = o.CustomerID
INNER JOIN OrderDetails od ON o. OrderID = od.OrderID
GROUP BY cu.CustomerName, ci.CityName, o.OrderID, o.OrderDate;

-- Query 3: Find customers who never placed an order
SELECT c.CustomerID, c.CustomerName
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID
WHERE o.OrderID IS NULL;

-- Query 4: Products that were never ordered
SELECT p.ProductID, p.ProductName
FROM Products p
```

```
LEFT JOIN OrderDetails od ON p.ProductID = od.ProductID
WHERE od.OrderDetailID IS NULL;

-- Query 5: Complex query with multiple joins and conditions
SELECT
    cu.CustomerName,
    ci. CityName,
    COUNT(DISTINCT o.OrderID) AS TotalOrders,
    COUNT(DISTINCT od. ProductID) AS UniqueProducts,
    SUM(od.Quantity * od.UnitPrice) AS TotalSpent
FROM Customers cu
INNER JOIN Cities ci ON cu. CityID = ci.CityID
LEFT JOIN Orders o ON cu.CustomerID = o.CustomerID
LEFT JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE o.OrderDate >= DATEADD(YEAR, -1, GETDATE())
GROUP BY cu.CustomerName, ci.CityName
HAVING SUM(od.Quantity * od.UnitPrice) > 1000
ORDER BY TotalSpent DESC;
```

Aggregate Functions

13. What are aggregate functions in SQL Server?

Answer:

Aggregate functions perform calculations on a set of values and return a single value.

Common Aggregate Functions:

```
-- COUNT - Count number of rows
SELECT COUNT(*) AS TotalEmployees FROM Employees;
SELECT COUNT(ManagerID) AS EmployeesWithManager FROM Employees;
SELECT COUNT(DISTINCT Department) AS UniqueDepartments FROM Employees;

-- SUM - Calculate sum
SELECT SUM(Salary) AS TotalSalary FROM Employees;
SELECT SUM(Quantity * UnitPrice) AS TotalRevenue FROM OrderDetails;

-- AVG - Calculate average
SELECT AVG(Salary) AS AverageSalary FROM Employees;
SELECT AVG(CAST(Salary AS FLOAT)) AS AverageSalary FROM Employees; -- More precise

-- MIN - Find minimum value
SELECT MIN(Salary) AS MinimumSalary FROM Employees;
SELECT MIN(HireDate) AS FirstHireDate FROM Employees;

-- MAX - Find maximum value
SELECT MAX(Salary) AS MaximumSalary FROM Employees;
SELECT MAX(OrderDate) AS LatestOrder FROM Orders;

-- Multiple aggregates in one query
SELECT
    COUNT(*) AS TotalEmployees,
    SUM(Salary) AS TotalSalary,
    AVG(Salary) AS AverageSalary,
    MIN(Salary) AS MinSalary,
    MAX(Salary) AS MaxSalary
FROM Employees;
```

```
-- Aggregate with WHERE
SELECT AVG(Salary) AS AvgITSalary
FROM Employees
WHERE Department = 'IT';

-- Aggregate with DISTINCT
SELECT
    COUNT(*) AS TotalOrders,
    COUNT(DISTINCT CustomerID) AS UniqueCustomers,
    SUM(Amount) AS TotalRevenue
FROM Orders;
```

String Aggregation (SQL Server 2017+):

```
-- STRING_AGG - Concatenate values
SELECT
    Department,
    STRING_AGG(Name, ', ') AS EmployeeNames
FROM Employees
GROUP BY Department;

-- Result: IT:  John, Jane, Bob
```

Statistical Functions:

```
-- STDEV - Standard Deviation
SELECT STDEV(Salary) AS SalaryStdDev FROM Employees;

-- VAR - Variance
SELECT VAR(Salary) AS SalaryVariance FROM Employees;
```

GROUP BY and HAVING

14. What is GROUP BY and how does it differ from HAVING?

Answer:

GROUP BY - Groups rows with same values into summary rows

HAVING - Filters groups (used with GROUP BY)

```
-- Basic GROUP BY
SELECT Department, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department;

-- Result:
-- IT: 10
-- HR: 5
-- Finance: 8

-- GROUP BY with aggregate functions
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AvgSalary,
    MIN(Salary) AS MinSalary,
```

```
    MAX(Salary) AS MaxSalary
FROM Employees
GROUP BY Department;

-- WHERE vs HAVING
-- WHERE filters BEFORE grouping
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
WHERE HireDate >= '2020-01-01'  -- Filter individual rows
GROUP BY Department;

-- HAVING filters AFTER grouping
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 50000;  -- Filter groups

-- Combined WHERE and HAVING
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AvgSalary
FROM Employees
WHERE IsActive = 1  -- Filter:  Only active employees
GROUP BY Department
HAVING COUNT(*) > 5;  -- Filter: Departments with > 5 employees

-- GROUP BY multiple columns
SELECT
    Department,
    JobTitle,
    COUNT(*) AS Count,
    AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department, JobTitle
ORDER BY Department, JobTitle;

-- GROUP BY with JOIN
SELECT
    d.DepartmentName,
    COUNT(e.EmployeeID) AS EmployeeCount,
    AVG(e. Salary) AS AvgSalary
FROM Departments d
LEFT JOIN Employees e ON d.DepartmentID = e.DepartmentID
GROUP BY d.DepartmentName;

-- Complex example
SELECT
    YEAR(OrderDate) AS OrderYear,
    MONTH(OrderDate) AS OrderMonth,
    COUNT(DISTINCT CustomerID) AS UniqueCustomers,
    COUNT(OrderID) AS TotalOrders,
    SUM(TotalAmount) AS Revenue
FROM Orders
WHERE OrderDate >= DATEADD(YEAR, -2, GETDATE())
GROUP BY YEAR(OrderDate), MONTH(OrderDate)
HAVING SUM(TotalAmount) > 10000
ORDER BY OrderYear DESC, OrderMonth DESC;
```

Difference between WHERE and HAVING:

Feature	WHERE	HAVING
Purpose	Filter rows	Filter groups
Applied	Before GROUP BY	After GROUP BY
Use with	Individual columns	Aggregate functions
Example	WHERE Salary > 50000	HAVING AVG(Salary) > 50000

INSERT, UPDATE, DELETE

15. How do you insert, update, and delete data?

Answer:

INSERT Statement:

```
-- Insert single row
INSERT INTO Employees (EmployeeID, Name, Department, Salary)
VALUES (1, 'John Doe', 'IT', 60000);

-- Insert multiple rows
INSERT INTO Employees (EmployeeID, Name, Department, Salary)
VALUES
    (2, 'Jane Smith', 'HR', 55000),
    (3, 'Bob Johnson', 'IT', 65000),
    (4, 'Alice Williams', 'Finance', 58000);

-- Insert without specifying columns (all columns)
INSERT INTO Employees VALUES (5, 'Charlie Brown', 'IT', 62000, '2023-01-15');

-- Insert with default values
INSERT INTO Employees (EmployeeID, Name, Department)
VALUES (6, 'David Lee', 'HR');  -- Salary will be NULL or DEFAULT

-- Insert from SELECT (copy data)
INSERT INTO EmployeesBackup
SELECT * FROM Employees WHERE Department = 'IT';

-- Insert with specific columns from SELECT
INSERT INTO EmployeeSummary (Name, Department, Salary)
SELECT Name, Department, Salary
FROM Employees
WHERE HireDate >= '2023-01-01';

-- Insert into table with IDENTITY column
INSERT INTO Employees (Name, Department, Salary)
VALUES ('Emily Davis', 'Finance', 59000);  -- EmployeeID auto-generated
```

UPDATE Statement:

```
-- Update single column
UPDATE Employees
SET Salary = 70000
WHERE EmployeeID = 1;

-- Update multiple columns
UPDATE Employees
SET
```

```
    Salary = 75000,
    Department = 'Management'
WHERE EmployeeID = 1;

-- Update with calculation
UPDATE Employees
SET Salary = Salary * 1.10  -- 10% raise
WHERE Department = 'IT';

-- Update with JOIN
UPDATE e
SET e.DepartmentName = d.DepartmentName
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID;

-- Update with subquery
UPDATE Employees
SET Salary = (SELECT AVG(Salary) FROM Employees WHERE Department = 'IT')
WHERE EmployeeID = 10;

-- Update all rows (careful!)
UPDATE Employees
SET IsActive = 1;

-- Update with CASE
UPDATE Employees
SET Salary = CASE
    WHEN Department = 'IT' THEN Salary * 1.15
    WHEN Department = 'HR' THEN Salary * 1.10
    WHEN Department = 'Finance' THEN Salary * 1.12
    ELSE Salary
END;
```

DELETE Statement:

```
-- Delete specific rows
DELETE FROM Employees
WHERE EmployeeID = 1;

-- Delete with condition
DELETE FROM Employees
WHERE Department = 'IT' AND Salary < 50000;

-- Delete with subquery
DELETE FROM Employees
WHERE DepartmentID IN (SELECT DepartmentID FROM Departments WHERE DepartmentName = 'Old Dept');

-- Delete with JOIN
DELETE e
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE d.IsActive = 0;

-- Delete all rows (but keeps table structure)
DELETE FROM Employees;

-- TRUNCATE - Faster way to delete all rows
TRUNCATE TABLE Employees;  -- Cannot use WHERE clause

-- Delete with TOP
DELETE TOP (10) FROM Employees
WHERE Department = 'IT';
```

Difference between DELETE and TRUNCATE:

Feature	DELETE	TRUNCATE
Speed	Slower	Faster
WHERE clause	Yes	No
Triggers	Fires triggers	Does not fire triggers
Rollback	Can rollback	Can rollback (in transaction)
Identity	Doesn't reset	Resets identity
Logging	Logs each row	Minimal logging

Constraints

16. What are constraints in SQL Server?

Answer:

Constraints are rules enforced on data columns to ensure data integrity.

Types of Constraints:

1. PRIMARY KEY - Uniquely identifies each record

```
-- Single column primary key
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50)
);

-- Primary key with IDENTITY
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY IDENTITY(1,1),
    Name VARCHAR(50)
);

-- Composite primary key
CREATE TABLE OrderDetails
(
    OrderID INT,
    ProductID INT,
    Quantity INT,
    PRIMARY KEY (OrderID, ProductID)
);

-- Add primary key to existing table
ALTER TABLE Employees
ADD CONSTRAINT PK_Employees PRIMARY KEY (EmployeeID);
```

2. FOREIGN KEY - Links two tables together

```
CREATE TABLE Departments
(
    DepartmentID INT PRIMARY KEY,
```

```
    DepartmentName VARCHAR(50)
);

CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);

-- Foreign key with named constraint
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    DepartmentID INT,
    CONSTRAINT FK_Employees_Departments
        FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);

-- Foreign key with cascade
CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    CONSTRAINT FK_Orders_Customers
        FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
        ON DELETE CASCADE  -- Delete orders when customer is deleted
        ON UPDATE CASCADE  -- Update OrderID when CustomerID changes
);

-- Add foreign key to existing table
ALTER TABLE Employees
ADD CONSTRAINT FK_Employees_Departments
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID);
```

3. UNIQUE - Ensures all values in column are different

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE,
    SSN VARCHAR(11) UNIQUE
);

-- Named unique constraint
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Email VARCHAR(100),
    CONSTRAINT UQ_Employees_Email UNIQUE (Email)
);

-- Composite unique constraint
CREATE TABLE Products
(
    ProductID INT PRIMARY KEY,
    ProductCode VARCHAR(20),
    ProductName VARCHAR(100),
    CONSTRAINT UQ_Products_Code_Name UNIQUE (ProductCode, ProductName)
);
```

```
-- Add unique constraint to existing table
ALTER TABLE Employees
ADD CONSTRAINT UQ_Employees_Email UNIQUE (Email);
```

4. CHECK - Ensures value meets specific condition

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18 AND Age <= 65),
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    Email VARCHAR(100) CHECK (Email LIKE '%@%. %')
);

-- Named check constraint
CREATE TABLE Products
(
    ProductID INT PRIMARY KEY,
    Price DECIMAL(10,2),
    Stock INT,
    CONSTRAINT CHK_Products_Price CHECK (Price > 0),
    CONSTRAINT CHK_Products_Stock CHECK (Stock >= 0)
);

-- Check with multiple columns
CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,
    OrderDate DATE,
    ShipDate DATE,
    CONSTRAINT CHK_Orders_Dates CHECK (ShipDate >= OrderDate)
);

-- Add check constraint to existing table
ALTER TABLE Employees
ADD CONSTRAINT CHK_Employees_Age CHECK (Age >= 18);
```

5. DEFAULT - Sets default value for column

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    HireDate DATE DEFAULT GETDATE(),
    IsActive BIT DEFAULT 1,
    Department VARCHAR(50) DEFAULT 'General',
    Salary DECIMAL(10,2) DEFAULT 0
);

-- Named default constraint
CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,
    OrderDate DATE CONSTRAINT DF_Orders_OrderDate DEFAULT GETDATE(),
    Status VARCHAR(20) CONSTRAINT DF_Orders_Status DEFAULT 'Pending'
);

-- Add default constraint to existing table
ALTER TABLE Employees
ADD CONSTRAINT DF_Employees_HireDate DEFAULT GETDATE() FOR HireDate;
```

```
-- Drop default constraint
ALTER TABLE Employees
DROP CONSTRAINT DF_Employees_HireDate;
```

6. NOT NULL - Column cannot have NULL value

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    Phone VARCHAR(20)  -- Can be NULL
);

-- Add NOT NULL to existing column
ALTER TABLE Employees
ALTER COLUMN Phone VARCHAR(20) NOT NULL;

-- Remove NOT NULL
ALTER TABLE Employees
ALTER COLUMN Phone VARCHAR(20) NULL;
```

Managing Constraints:

```
-- Drop constraint
ALTER TABLE Employees
DROP CONSTRAINT FK_Employees_Departments;

-- Disable constraint (temporarily)
ALTER TABLE Employees
NOCHECK CONSTRAINT FK_Employees_Departments;

-- Enable constraint
ALTER TABLE Employees
CHECK CONSTRAINT FK_Employees_Departments;

-- Disable all constraints on table
ALTER TABLE Employees
NOCHECK CONSTRAINT ALL;

-- Enable all constraints on table
ALTER TABLE Employees
CHECK CONSTRAINT ALL;

-- List all constraints on a table
SELECT
    CONSTRAINT_NAME,
    CONSTRAINT_TYPE
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE TABLE_NAME = 'Employees';

-- Get constraint definition
EXEC sp_helpconstraint 'Employees';
```

Views

17. What is a View and how do you create it?

Answer:

A View is a virtual table based on the result of a SELECT query.

Benefits:

- Simplify complex queries
- Security (hide columns)
- Reusability
- Logical data independence

```
-- Create simple view
CREATE VIEW vw_EmployeeList AS
SELECT EmployeeID, FirstName, LastName, Department
FROM Employees;

-- Query the view
SELECT * FROM vw_EmployeeList;

-- Create view with WHERE clause
CREATE VIEW vw_ITEmployees AS
SELECT EmployeeID, FirstName, LastName, Salary
FROM Employees
WHERE Department = 'IT';

-- Create view with JOIN
CREATE VIEW vw_EmployeeDepartments AS
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS FullName,
    d.DepartmentName,
    e.Salary
FROM Employees e
INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID;

-- Create view with aggregate functions
CREATE VIEW vw_DepartmentSummary AS
SELECT
    Department,
    COUNT(*) AS EmployeeCount,
    AVG(Salary) AS AverageSalary,
    MIN(Salary) AS MinSalary,
    MAX(Salary) AS MaxSalary
FROM Employees
GROUP BY Department;

-- Create view with calculations
CREATE VIEW vw_EmployeeWithBonus AS
SELECT
    EmployeeID,
    FirstName,
    LastName,
    Salary,
    Salary * 0.10 AS Bonus,
    Salary * 1.10 AS TotalCompensation
FROM Employees;

-- Alter (modify) view
ALTER VIEW vw_EmployeeList AS
SELECT EmployeeID, FirstName, LastName, Department, Email
```

```
FROM Employees
WHERE IsActive = 1;

-- Drop view
DROP VIEW vw_EmployeeList;

-- Drop view if exists
DROP VIEW IF EXISTS vw_EmployeeList;

-- Create or alter view (SQL Server 2016+)
CREATE OR ALTER VIEW vw_EmployeeList AS
SELECT EmployeeID, FirstName, LastName, Department
FROM Employees;

-- View with SCHEMABINDING (prevents base table changes)
CREATE VIEW vw_EmployeeNames
WITH SCHEMABINDING
AS
SELECT EmployeeID, FirstName, LastName
FROM dbo.Employees;

-- View with ENCRYPTION (hides definition)
CREATE VIEW vw_SalaryInfo
WITH ENCRYPTION
AS
SELECT EmployeeID, Salary
FROM Employees;

-- Indexed View (Materialized View)
CREATE VIEW vw_OrderSummary
WITH SCHEMABINDING
AS
SELECT
    CustomerID,
    COUNT_BIG(*) AS OrderCount,
    SUM(TotalAmount) AS TotalRevenue
FROM dbo.Orders
GROUP BY CustomerID;

-- Create unique clustered index on view
CREATE UNIQUE CLUSTERED INDEX IX_vw_OrderSummary
ON vw_OrderSummary (CustomerID);

-- Check if view exists
IF OBJECT_ID('vw_EmployeeList', 'V') IS NOT NULL
    DROP VIEW vw_EmployeeList;

-- Get view definition
EXEC sp_helptext 'vw_EmployeeList';

-- List all views
SELECT * FROM INFORMATION_SCHEMA.VIEWS;
```

Updatable Views:

```
-- Create updatable view
CREATE VIEW vw_EmployeeEdit AS
SELECT EmployeeID, FirstName, LastName, Email
FROM Employees
WHERE Department = 'IT';

-- Insert through view
```

```
INSERT INTO vw_EmployeeEdit (EmployeeID, FirstName, LastName, Email)
VALUES (101, 'John', 'Doe', 'john@example. com');

-- Update through view
UPDATE vw_EmployeeEdit
SET Email = 'newemail@example.com'
WHERE EmployeeID = 101;

-- Delete through view
DELETE FROM vw_EmployeeEdit
WHERE EmployeeID = 101;

-- WITH CHECK OPTION (ensures updates meet view criteria)
CREATE VIEW vw_ITEmployeesOnly AS
SELECT EmployeeID, FirstName, LastName, Department
FROM Employees
WHERE Department = 'IT'
WITH CHECK OPTION;

-- This will fail because Department must be 'IT'
INSERT INTO vw_ITEmployeesOnly (EmployeeID, FirstName, LastName, Department)
VALUES (102, 'Jane', 'Smith', 'HR');  -- Error!
```

Indexes

18. What are indexes and why are they important?

Answer:

An index is a database object that improves the speed of data retrieval operations.

Types of Indexes:

1. Clustered Index - Sorts and stores data rows in the table

```
-- Create clustered index (only one per table)
CREATE CLUSTERED INDEX IX_Employees_EmployeeID
ON Employees (EmployeeID);

-- Primary key creates clustered index by default
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,  -- Automatically creates clustered index
    Name VARCHAR(50)
);

-- Create table with explicit clustered index
CREATE TABLE Products
(
    ProductID INT PRIMARY KEY NONCLUSTERED,
    ProductCode VARCHAR(20),
    ProductName VARCHAR(100)
);

CREATE CLUSTERED INDEX IX_Products_ProductCode
ON Products (ProductCode);
```

2. Non-Clustered Index - Creates a separate structure with pointers

```
-- Create non-clustered index on single column
CREATE NONCLUSTERED INDEX IX_Employees_LastName
ON Employees (LastName);

-- Create non-clustered index on multiple columns
CREATE NONCLUSTERED INDEX IX_Employees_Dept_Salary
ON Employees (Department, Salary);

-- Create unique non-clustered index
CREATE UNIQUE NONCLUSTERED INDEX IX_Employees_Email
ON Employees (Email);

-- Create index with included columns (covering index)
CREATE NONCLUSTERED INDEX IX_Employees_LastName_Include
ON Employees (LastName)
INCLUDE (FirstName, Email, Department);

-- Create filtered index (SQL Server 2008+)
CREATE NONCLUSTERED INDEX IX_Employees_Active
ON Employees (LastName)
WHERE IsActive = 1;
```

3. Unique Index - Ensures unique values

```
CREATE UNIQUE INDEX IX_Employees_SSN
ON Employees (SSN);
```

Managing Indexes:

```
-- Drop index
DROP INDEX IX_Employees_LastName ON Employees;

-- Rebuild index (defragment)
ALTER INDEX IX_Employees_LastName ON Employees REBUILD;

-- Reorganize index
ALTER INDEX IX_Employees_LastName ON Employees REORGANIZE;

-- Disable index
ALTER INDEX IX_Employees_LastName ON Employees DISABLE;

-- Enable index (rebuild required)
ALTER INDEX IX_Employees_LastName ON Employees REBUILD;

-- Rebuild all indexes on table
ALTER INDEX ALL ON Employees REBUILD;

-- Get index information
EXEC sp_helpindex 'Employees';

-- View index fragmentation
SELECT
    OBJECT_NAME(ips.object_id) AS TableName,
    i.name AS IndexName,
    ips.avg_fragmentation_in_percent,
    ips.page_count
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'DETAILED') ips
INNER JOIN sys.indexes i ON ips.object_id = i.object_id
    AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 10
ORDER BY ips.avg_fragmentation_in_percent DESC;
```

```
-- List all indexes in database
SELECT
    t.name AS TableName,
    i.name AS IndexName,
    i.type_desc AS IndexType,
    i.is_unique,
    i.is_primary_key
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
WHERE i.name IS NOT NULL
ORDER BY t.name, i.name;
```

When to Use Indexes:

✅ **Create Index When:**

- Column is frequently used in WHERE clause
- Column is used in JOIN operations
- Column is used in ORDER BY
- Column has high selectivity (many unique values)
- Table is large

❌ **Avoid Index When:**

- Table is small
- Column has frequent INSERT/UPDATE/DELETE operations
- Column has low selectivity (few unique values)
- Column data type is large (text, image, etc.)

Example:

```
-- Without index (slow)
SELECT * FROM Employees WHERE LastName = 'Smith';  -- Table scan

-- Create index
CREATE INDEX IX_Employees_LastName ON Employees (LastName);

-- With index (fast)
SELECT * FROM Employees WHERE LastName = 'Smith';  -- Index seek
```

Stored Procedures

19. What are stored procedures and how do you create them?

Answer:

A stored procedure is a prepared SQL code that you can save and reuse.

Benefits:

- Performance (precompiled)
- Reusability
- Security

- Reduced network traffic
- Easier maintenance

```
-- Simple stored procedure
CREATE PROCEDURE sp_GetAllEmployees
AS
BEGIN
    SELECT * FROM Employees;
END;

-- Execute stored procedure
EXEC sp_GetAllEmployees;
-- or
EXECUTE sp_GetAllEmployees;

-- Stored procedure with INPUT parameter
CREATE PROCEDURE sp_GetEmployeeByID
    @EmployeeID INT
AS
BEGIN
    SELECT * FROM Employees WHERE EmployeeID = @EmployeeID;
END;

-- Execute with parameter
EXEC sp_GetEmployeeByID 101;
EXEC sp_GetEmployeeByID @EmployeeID = 101;

-- Stored procedure with multiple parameters
CREATE PROCEDURE sp_GetEmployeesByDeptAndSalary
    @Department VARCHAR(50),
    @MinSalary DECIMAL(10,2)
AS
BEGIN
    SELECT * FROM Employees
    WHERE Department = @Department
    AND Salary >= @MinSalary;
END;

-- Execute
EXEC sp_GetEmployeesByDeptAndSalary 'IT', 50000;
EXEC sp_GetEmployeesByDeptAndSalary @Department = 'IT', @MinSalary = 50000;

-- Stored procedure with default parameter value
CREATE PROCEDURE sp_GetEmployeesByDepartment
    @Department VARCHAR(50) = 'IT'  -- Default value
AS
BEGIN
    SELECT * FROM Employees WHERE Department = @Department;
END;

-- Execute with default
EXEC sp_GetEmployeesByDepartment;  -- Uses 'IT'
EXEC sp_GetEmployeesByDepartment 'HR';  -- Uses 'HR'

-- Stored procedure with OUTPUT parameter
CREATE PROCEDURE sp_GetEmployeeCount
    @Department VARCHAR(50),
    @EmployeeCount INT OUTPUT
AS
BEGIN
    SELECT @EmployeeCount = COUNT(*)
    FROM Employees
```

```
    WHERE Department = @Department;
END;

-- Execute with OUTPUT
DECLARE @Count INT;
EXEC sp_GetEmployeeCount 'IT', @Count OUTPUT;
PRINT 'Employee Count: ' + CAST(@Count AS VARCHAR);

-- Stored procedure with RETURN value
CREATE PROCEDURE sp_ValidateEmployee
    @EmployeeID INT
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Employees WHERE EmployeeID = @EmployeeID)
        RETURN 1;  -- Found
    ELSE
        RETURN 0;  -- Not found
END;

-- Execute and get return value
DECLARE @ReturnValue INT;
EXEC @ReturnValue = sp_ValidateEmployee 101;
IF @ReturnValue = 1
    PRINT 'Employee found';
ELSE
    PRINT 'Employee not found';

-- Stored procedure with INSERT
CREATE PROCEDURE sp_AddEmployee
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50),
    @Department VARCHAR(50),
    @Salary DECIMAL(10,2),
    @NewEmployeeID INT OUTPUT
AS
BEGIN
    INSERT INTO Employees (FirstName, LastName, Department, Salary)
    VALUES (@FirstName, @LastName, @Department, @Salary);

    SET @NewEmployeeID = SCOPE_IDENTITY();  -- Get newly created ID
END;

-- Execute
DECLARE @NewID INT;
EXEC sp_AddEmployee 'John', 'Doe', 'IT', 60000, @NewID OUTPUT;
PRINT 'New Employee ID: ' + CAST(@NewID AS VARCHAR);

-- Stored procedure with UPDATE
CREATE PROCEDURE sp_UpdateEmployeeSalary
    @EmployeeID INT,
    @NewSalary DECIMAL(10,2)
AS
BEGIN
    UPDATE Employees
    SET Salary = @NewSalary
    WHERE EmployeeID = @EmployeeID;

    IF @@ROWCOUNT = 0
        RETURN 0;  -- Not found
    ELSE
        RETURN 1;  -- Updated
END;
```

```
-- Stored procedure with DELETE
CREATE PROCEDURE sp_DeleteEmployee
    @EmployeeID INT
AS
BEGIN
    DELETE FROM Employees WHERE EmployeeID = @EmployeeID;

    RETURN @@ROWCOUNT;  -- Return number of rows deleted
END;

-- Stored procedure with error handling
CREATE PROCEDURE sp_TransferEmployee
    @EmployeeID INT,
    @NewDepartment VARCHAR(50)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        UPDATE Employees
        SET Department = @NewDepartment
        WHERE EmployeeID = @EmployeeID;

        IF @@ROWCOUNT = 0
            THROW 50001, 'Employee not found', 1;

        -- Log the transfer
        INSERT INTO EmployeeTransferLog (EmployeeID, TransferDate, NewDepartment)
        VALUES (@EmployeeID, GETDATE(), @NewDepartment);

        COMMIT TRANSACTION;
        RETURN 1;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;

        -- Return error information
        SELECT
            ERROR_NUMBER() AS ErrorNumber,
            ERROR_MESSAGE() AS ErrorMessage;

        RETURN 0;
    END CATCH
END;

-- Alter stored procedure
ALTER PROCEDURE sp_GetAllEmployees
AS
BEGIN
    SELECT EmployeeID, FirstName, LastName, Department, Salary
    FROM Employees
    WHERE IsActive = 1;
END;

-- Drop stored procedure
DROP PROCEDURE sp_GetAllEmployees;

-- Drop if exists (SQL Server 2016+)
DROP PROCEDURE IF EXISTS sp_GetAllEmployees;

-- List all stored procedures
```

```
SELECT name, create_date, modify_date
FROM sys.procedures;

-- Get stored procedure definition
EXEC sp_helptext 'sp_GetAllEmployees';

-- Get stored procedure parameters
EXEC sp_help 'sp_GetEmployeeByID';
```

Advanced Example:

```
CREATE PROCEDURE sp_ProcessOrder
    @CustomerID INT,
    @ProductID INT,
    @Quantity INT,
    @OrderID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;  -- Don't return row count

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Check product stock
        DECLARE @Stock INT;
        SELECT @Stock = Stock FROM Products WHERE ProductID = @ProductID;

        IF @Stock < @Quantity
        BEGIN
            THROW 50002, 'Insufficient stock', 1;
        END

        -- Create order
        INSERT INTO Orders (CustomerID, OrderDate, TotalAmount)
        VALUES (@CustomerID, GETDATE(), 0);

        SET @OrderID = SCOPE_IDENTITY();

        -- Add order details
        DECLARE @UnitPrice DECIMAL(10,2);
        SELECT @UnitPrice = Price FROM Products WHERE ProductID = @ProductID;

        INSERT INTO OrderDetails (OrderID, ProductID, Quantity, UnitPrice)
        VALUES (@OrderID, @ProductID, @Quantity, @UnitPrice);

        -- Update order total
        UPDATE Orders
        SET TotalAmount = @Quantity * @UnitPrice
        WHERE OrderID = @OrderID;

        -- Update product stock
        UPDATE Products
        SET Stock = Stock - @Quantity
        WHERE ProductID = @ProductID;

        COMMIT TRANSACTION;
        RETURN 1;  -- Success
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
```

```
        THROW;  -- Re-throw error
        RETURN 0;  -- Failure
    END CATCH
END;
```

Functions

20. What are the different types of functions in SQL Server?

Answer:

1. Scalar Functions - Return single value

```
-- Create scalar function
CREATE FUNCTION fn_GetEmployeeFullName
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50)
)
RETURNS VARCHAR(100)
AS
BEGIN
    RETURN @FirstName + ' ' + @LastName;
END;

-- Use scalar function
SELECT dbo.fn_GetEmployeeFullName(FirstName, LastName) AS FullName
FROM Employees;

-- Scalar function with calculation
CREATE FUNCTION fn_CalculateAge
(
    @BirthDate DATE
)
RETURNS INT
AS
BEGIN
    RETURN DATEDIFF(YEAR, @BirthDate, GETDATE());
END;

-- Use function
SELECT
    Name,
    BirthDate,
    dbo.fn_CalculateAge(BirthDate) AS Age
FROM Employees;

-- Scalar function with conditional logic
CREATE FUNCTION fn_GetTaxRate
(
    @Salary DECIMAL(10,2)
)
RETURNS DECIMAL(5,2)
AS
BEGIN
    DECLARE @TaxRate DECIMAL(5,2);

    IF @Salary < 30000
        SET @TaxRate = 0.10;
```

```
    ELSE IF @Salary < 60000
        SET @TaxRate = 0.20;
    ELSE
        SET @TaxRate = 0.30;

    RETURN @TaxRate;
END;
```

2. Table-Valued Functions - Return table

Inline Table-Valued Function:

```
-- Create inline table-valued function
CREATE FUNCTION fn_GetEmployeesByDepartment
(
    @Department VARCHAR(50)
)
RETURNS TABLE
AS
RETURN
(
    SELECT EmployeeID, FirstName, LastName, Salary
    FROM Employees
    WHERE Department = @Department
);

-- Use table-valued function
SELECT * FROM dbo.fn_GetEmployeesByDepartment('IT');

-- Join with table-valued function
SELECT e.*, d.DepartmentName
FROM dbo.fn_GetEmployeesByDepartment('IT') e
INNER JOIN Departments d ON e.Department = d.DepartmentName;
```

Multi-Statement Table-Valued Function:

```
-- Create multi-statement table-valued function
CREATE FUNCTION fn_GetEmployeeSummary
(
    @MinSalary DECIMAL(10,2)
)
RETURNS @EmployeeTable TABLE
(
    EmployeeID INT,
    FullName VARCHAR(100),
    Department VARCHAR(50),
    Salary DECIMAL(10,2),
    TaxAmount DECIMAL(10,2)
)
AS
BEGIN
    INSERT INTO @EmployeeTable
    SELECT
        EmployeeID,
        FirstName + ' ' + LastName,
        Department,
        Salary,
        Salary * 0.20  -- 20% tax
    FROM Employees
    WHERE Salary >= @MinSalary;
```

```
    RETURN;
END;

-- Use function
SELECT * FROM dbo.fn_GetEmployeeSummary(50000);
```

Built-in Functions:

```
-- String Functions
SELECT
    LEN('Hello') AS Length,                    -- 5
    UPPER('hello') AS Uppercase,               -- HELLO
    LOWER('HELLO') AS Lowercase,               -- hello
    LTRIM('  hello  ') AS LeftTrim,           -- 'hello  '
    RTRIM('  hello  ') AS RightTrim,          -- '  hello'
    TRIM('  hello  ') AS Trim,                 -- 'hello'
    SUBSTRING('Hello World', 1, 5) AS Sub,     -- Hello
    REPLACE('Hello', 'l', 'x') AS Replace,     -- Hexxo
    CHARINDEX('o', 'Hello') AS Position,       -- 5
    CONCAT('Hello', ' ', 'World') AS Concat,   -- Hello World
    LEFT('Hello', 3) AS Left,                  -- Hel
    RIGHT('Hello', 3) AS Right,                -- llo
    REVERSE('Hello') AS Reverse;               -- olleH

-- Date Functions
SELECT
    GETDATE() AS CurrentDateTime,
    GETUTCDATE() AS UTCDateTime,
    DATEADD(DAY, 7, GETDATE()) AS NextWeek,
    DATEDIFF(DAY, '2023-01-01', GETDATE()) AS DaysSince,
    YEAR(GETDATE()) AS Year,
    MONTH(GETDATE()) AS Month,
    DAY(GETDATE()) AS Day,
    DATENAME(WEEKDAY, GETDATE()) AS WeekDay,
    DATEPART(HOUR, GETDATE()) AS Hour,
    EOMONTH(GETDATE()) AS EndOfMonth,
    DATEFROMPARTS(2023, 12, 25) AS ChristmasDate;

-- Math Functions
SELECT
    ABS(-10) AS Absolute,                      -- 10
    CEILING(10.3) AS Ceiling,                  -- 11
    FLOOR(10.8) AS Floor,                      -- 10
    ROUND(10.567, 2) AS Round,                 -- 10.57
    POWER(2, 3) AS Power,                      -- 8
    SQRT(16) AS SquareRoot,                    -- 4
    RAND() AS Random,                          -- Random 0-1
    SIGN(-10) AS Sign;                         -- -1

-- Aggregate Functions
SELECT
    COUNT(*) AS TotalRows,
    SUM(Salary) AS TotalSalary,
    AVG(Salary) AS AverageSalary,
    MIN(Salary) AS MinSalary,
    MAX(Salary) AS MaxSalary
FROM Employees;

-- Conversion Functions
SELECT
    CAST('123' AS INT) AS CastToInt,
    CONVERT(VARCHAR, GETDATE(), 101) AS ConvertDate,  -- MM/DD/YYYY
```

```
    TRY_CAST('abc' AS INT) AS TryCast,        -- NULL (no error)
    TRY_CONVERT(INT, 'abc') AS TryConvert;    -- NULL (no error)

-- NULL Functions
SELECT
    ISNULL(NULL, 'Default') AS IsNull,         -- Default
    COALESCE(NULL, NULL, 'First Non-NULL') AS Coalesce,  -- First Non-NULL
    NULLIF(10, 10) AS NullIf;                  -- NULL (equal values)

-- System Functions
SELECT
    @@VERSION AS SQLVersion,
    @@SERVERNAME AS ServerName,
    DB_NAME() AS CurrentDatabase,
    USER_NAME() AS CurrentUser,
    HOST_NAME() AS HostName,
    @@ROWCOUNT AS RowCount,
    SCOPE_IDENTITY() AS LastIdentity,
    NEWID() AS NewGUID;
```

Managing Functions:

```
-- Alter function
ALTER FUNCTION fn_GetEmployeeFullName
(
    @FirstName VARCHAR(50),
    @LastName VARCHAR(50)
)
RETURNS VARCHAR(100)
AS
BEGIN
    RETURN UPPER(@FirstName + ' ' + @LastName);
END;

-- Drop function
DROP FUNCTION fn_GetEmployeeFullName;

-- Drop if exists (SQL Server 2016+)
DROP FUNCTION IF EXISTS fn_GetEmployeeFullName;

-- List all functions
SELECT name, type_desc
FROM sys.objects
WHERE type IN ('FN', 'IF', 'TF');  -- FN=Scalar, IF=Inline, TF=Table

-- Get function definition
EXEC sp_helptext 'fn_GetEmployeeFullName';
```

Triggers

21. What are triggers and what are the different types?

Answer:

A trigger is a special type of stored procedure that automatically executes when an event occurs.

Types of Triggers:

1. DML Triggers - Fire on INSERT, UPDATE, DELETE

```
-- AFTER INSERT Trigger
CREATE TRIGGER tr_AfterInsert_Employee
ON Employees
AFTER INSERT
AS
BEGIN
    PRINT 'New employee added';

    -- Insert into audit table
    INSERT INTO EmployeeAudit (EmployeeID, Action, ActionDate)
    SELECT EmployeeID, 'INSERT', GETDATE()
    FROM inserted;
END;

-- AFTER UPDATE Trigger
CREATE TRIGGER tr_AfterUpdate_Employee
ON Employees
AFTER UPDATE
AS
BEGIN
    -- Log salary changes
    INSERT INTO SalaryChangeLog (EmployeeID, OldSalary, NewSalary, ChangeDate)
    SELECT
        i.EmployeeID,
        d. Salary AS OldSalary,
        i.Salary AS NewSalary,
        GETDATE()
    FROM inserted i
    INNER JOIN deleted d ON i.EmployeeID = d.EmployeeID
    WHERE i.Salary <> d.Salary;
END;

-- AFTER DELETE Trigger
CREATE TRIGGER tr_AfterDelete_Employee
ON Employees
AFTER DELETE
AS
BEGIN
    -- Archive deleted employees
    INSERT INTO EmployeeArchive
    SELECT *, GETDATE() AS DeletedDate
    FROM deleted;
END;

-- INSTEAD OF Trigger (replaces the operation)
CREATE TRIGGER tr_InsteadOfDelete_Employee
ON Employees
INSTEAD OF DELETE
AS
BEGIN
    -- Soft delete instead of actual delete
    UPDATE Employees
    SET IsActive = 0, DeletedDate = GETDATE()
    WHERE EmployeeID IN (SELECT EmployeeID FROM deleted);
END;

-- Multiple events trigger
CREATE TRIGGER tr_Employee_AuditAll
```